

The Bombay Salesian Society's
DON BOSCO INSTITUTE OF TECHNOLOGY, Kurla(W)
Premier Automobiles Road, Kurla (W), Mumbai-70

**Approved by AICTE, Govt. of Maharashtra
&
Affiliated to the University of Mumbai**



**S.E MINI PROJECT REPORT
CSM401 - Mini Project 1B**

On

ReadySetPlay

Department of computer Engineering

University of Mumbai

April 2025



The Bombay Salesian Society's
DON BOSCO INSTITUTE OF TECHNOLOGY, Kurla(W)
Premier Automobiles Road, Kurla (W), Mumbai-70

Department of Computer Engineering
(Session 2024-2025 EVEN Semester)

CERTIFICATE

Project Title: ReadySetPlay – A Performance Companion

Project Team Members:

1. Anton Jarvis	04
2. Keigan Merv Cardoza	12
3. Christopher Chettiar	13

Name of Internal Guide(s): 1. Prof. Kalpita Ajinkya Wagaskar
2. Prof Seema Nehete

INTERNAL EXAMINER (s)

EXTERNAL EXAMINER (s)

HEAD, COMPUTER ENGINEERING

Date: 8th April 2025

ABSTRACT

ReadySetPlay is a performance companion Android application designed to assist musicians during live performances by providing real-time chord detection, lyrics and chords display, and tempo-synced auto-scrolling. The need for such an app arises from the challenges musicians face while managing setlists, chord sheets, and smooth transitions during performances.

The purpose of this project is to simplify on-stage performance workflows by integrating features such as chord transposition, a full-featured text editor, setlist creation, and live chord recognition.

The application was developed using Kotlin for Android, Retrofit for song data retrieval. Lyrics and chords are fetched from self-created API and stored locally using a Room database.

Testing revealed effective real-time chord detection with smooth scrolling of lyrics, as well as reliable file operations including save, open, and create new song sheets. Transposition and styling options enhanced the app's usability for live performances.

In conclusion, ReadySetPlay successfully bridges the gap between traditional songbooks and intelligent digital performance tools, offering a smart and intuitive interface for performing artists.

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Certificate	ii
2	Table of Contents	iii
3	Table of Figures	iv
4	Table of Tables	v
5	Abbreviations	vi
Chapter 1	Introduction	1
Chapter 2	Literature Survey	2
2.1	Survey of existing systems	2
2.2	Limitation of existing system	4
2.3	Problem statement and objective	5
2.4	Scope	5
Chapter 3	Proposed System	6
3.1	Analysis / Framework / Algorithm	6
3.2	Details of hardware and software	7
3.3	Design details	8
3.4	Methodology	9
Chapter 4	Implementation details	10
4.1	Implementation Discussion	11
Chapter 5	Results and Discussions	12
5.1	Conclusion	13
5.2	References	14
5.3	Appendix	15
5.4	Acknowledgement	17

TABLE OF FIGURES

Fig. No.	Figure Caption	Page No.
1	App Architecture	8

TABLE OF TABLES

Table No.	Table Name	Page No.
1	Table of Contents	iii
2	Table of Figures	iv
3	Abbreviations	vi
4	Hardware Requirements	7
5	Software Requirements	7
6	Module Wise Design	8
7	Activity Gantt	10

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
DB	Database
MySQL	My Structured Query Language
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
DBIT	Don Bosco Institute of Technology
HTTP	HyperText Transfer Protocol
URL	Uniform Resource Locator
XML	eXtensible Markup Language
SDK	Software Development Kit
UG	Ultimate Guitar
UML	Unified Modeling Language
CRUD	Create, Read, Update, Delete
SE	Second Year
T.E.	Third Year Engineering
MIT	Massachusetts Institute of Technology
OCR	Optical Character Recognition (if mentioned in extension)
GCP	Google Cloud Platform (if used)
REST	Representational State Transfer
CPU	Central Processing Unit
RAM	Random Access Memory

Chapter 1: INTRODUCTION

Live musical performance has always required a delicate balance between artistic expression and technical preparedness. Over the years, digital tools have increasingly been integrated into the performance process to assist musicians with lyrics, chords, and setlist management. Applications such as OnSong, SongSheet Pro, and Ultimate Guitar offer static song displays, chord transposition, and setlist functionalities. While these apps offer utility, they often lack dynamic interaction with live performance inputs, such as audio-based feedback or real-time chord recognition.

Recent studies and tools in the field of music technology have explored the use of artificial intelligence and signal processing to support musicians more actively. Real-time audio analysis and chord detection libraries like TarsosDSP have shown promising results in recognizing musical patterns and pitches, which can be leveraged for performance automation. However, the integration of these tools into a cohesive mobile application that directly enhances live performance remains limited. Many existing applications rely on manual control and provide minimal interactivity, which can be distracting during live shows.

One major area for improvement is the ability to synchronize the display of lyrics and chords with the actual music being played. If an app could detect chords in real time and adjust the on-screen content accordingly, it would significantly reduce the cognitive load on performers. Additionally, the ability to stylize, transpose, and manage content within a single, streamlined interface is still not fully realized in existing solutions.

This project hypothesizes that integrating real-time chord detection with automatic lyric scrolling and text editing capabilities into a mobile app will significantly enhance the efficiency and quality of live music performances. To test this hypothesis, an Android application called **ReadySetPlay** was developed using Kotlin. It integrates chord and lyric scraping via APIs, local song storage using Room, and live chord detection through TarsosDSP. It also features a rich text editor with file management and chord transposition capabilities.

The outcome of this project aims to contribute to the growing field of intelligent music performance tools. By combining machine listening with performance-friendly UI design, ReadySetPlay offers an innovative approach to supporting musicians in real time, paving the way for smarter digital performance companions.

Chapter 2: LITERATURE SURVEY

This chapter critically examines recent advancements in music-related app development and technology integration relevant to your Android application **ReadySetPlay**. The review focuses on methods for lyric and chord extraction, real-time processing, and music transcription technologies, primarily based on recent research literature.

2.1 Reviewed Systems

1. SongTrans: A Unified Song Transcription and Alignment Method for Lyrics and Notes

Siwei Wu, Jinzheng He, Ruibin Yuan, et al., 2024 – IEEE Transactions on Audio, Speech, and Language Processing

SongTrans introduces a comprehensive system that enables the simultaneous transcription of both lyrics and musical notes from audio recordings. It employs a novel dual-pathway architecture, using autoregressive models to handle the sequential nature of lyrics and duration estimation, while applying non-autoregressive models for efficient pitch and note prediction. This results in an efficient end-to-end pipeline capable of aligning musical structure with corresponding lyrical components without needing extensive pre-labeled datasets.

- **Strengths:**

The architecture is innovative due to its hybrid prediction mechanism, which allows accurate alignment of both lyrics and musical notes. This approach significantly reduces preprocessing time and enhances generalization across genres and audio qualities.

2. LyricJam: A System for Generating Lyrics for Live Instrumental Music

Olga Vechtomova, Gaurav Sahu, Dhruv Kumar, 2021 – IEEE Transactions on Affective Computing

LyricJam is an AI-powered system designed to inspire musicians during jam sessions by generating contextually relevant lyrics based on real-time instrumental input. It works by extracting high-level audio features—such as rhythm, harmony, and tonal changes—and embedding them into a latent space. A generative adversarial network (GAN) then maps these features to text sequences that reflect the emotional and tonal character of the performance.

- **Strengths:**

The system's real-time nature and context sensitivity make it a valuable tool for artists seeking spontaneous lyric inspiration during live or creative sessions. It emphasizes co-creativity between humans and machines, enhancing improvisation and artistic exploration.

3. Chordify: Chord Extraction and Alignment for Online Music Learning

Johannes Van der Wees, Bas De Haas, 2019 – Proceedings of ISMIR

Chordify is an online platform that analyzes uploaded audio tracks to extract and synchronize chords with the rhythm of the song. It uses beat-synchronous chroma features along with hidden Markov models to estimate chord transitions accurately, aligning them with the song timeline to aid in music practice and learning.

- **Strengths:**

Chordify's robust chord recognition algorithms have made it a widely trusted tool among learners and hobbyists. The beat alignment enhances user experience by synchronizing chord progression with playback in a user-friendly manner. It supports various musical genres and integrates playback tools for easier learning.

4. TarsosDSP: A Real-Time Audio Processing Framework for Java

Jeroen B. L. Six, 2015 – Journal of Open Research Software

TarsosDSP is a Java-based digital signal processing (DSP) library that provides real-time tools for audio analysis, including pitch detection, beat tracking, time stretching, and filtering. It has been extensively used in both research and mobile music applications and is notable for its compatibility with Android development environments.

- **Strengths:**

It is highly modular and supports low-latency audio processing, making it well-suited for real-time musical interactions, such as instrument tuning, pitch monitoring, and onset detection. TarsosDSP is lightweight and integrates efficiently with Kotlin-based Android apps like ReadySetPlay.

5. Interactive Mobile Applications for Music Practice: A Review and Taxonomy

Lisa Fujikawa, Carlos Agon, Jean Bresson, 2022 – SpringerLink: Journal of New Music Research

This paper presents a detailed review of mobile applications designed to support music practice. It categorizes apps based on their function—learning, performance, and composition—and examines their core features such as interactive feedback, real-time visual aids, and practice loop creation. The authors also propose a taxonomy to analyze the design trends, technical implementation, and educational impact of mobile music tools.

- **Strengths:**

The paper offers valuable insight into the design and technological components of effective music practice apps. It highlights the need for features like user control, visual feedback, personalization, and multi-functionality, which align closely with the aims of ReadySetPlay.

2.2 Limitations of Reviewed Systems

While the reviewed systems provide valuable contributions to the fields of music transcription, lyric generation, and performance assistance, several key limitations prevent them from serving as comprehensive solutions for real-time stage use or unified performance preparation:

1. **SongTrans**
 - Lacks mobile compatibility or deployment options for Android devices.
 - Not suitable for live, real-time usage or editing/transposition workflows.
 - Focused primarily on academic transcription with no user interface for performers.
2. **LyricJam**
 - Does not support chord recognition or editing features.
 - Lacks structural lyric formatting or setlist creation.
 - Not available on mobile platforms, and unsuitable for performance preparation or live execution.
3. **Chordify**
 - Web-based only; no Android or offline functionality.
 - No support for lyrics, editable song sheets, or chord transposition.
 - Not designed for live performance needs or real-time interaction with musicians.
4. **TarsosDSP**
 - Does not provide native chord recognition—only pitch detection.
 - Requires custom implementation for chord interpretation and UI visualization.
 - Minimal frontend tools; unsuitable for direct end-user interaction without significant development effort.
5. **Review and Taxonomy of Music Practice Apps**
 - Focuses on categorization and theory without implementing a unified tool.
 - Lacks detailed technical solutions for chord handling or lyric-based interfaces.
 - No real-time or mobile-first application examples covering all core features in one system.

2.3 Problem Statement and Objectives

Despite the availability of numerous mobile applications designed for musicians, most of these tools cater to isolated functionalities such as chord recognition, lyric viewing, or setlist creation. They often lack integration and real-time interactivity, which are crucial during live performances. Musicians, especially guitarists and vocalists, face challenges in managing lyrics, chords, and performance tools in a single application. This results in interrupted workflows, inefficient stage setups, and increased dependency on multiple devices or applications.

Currently, there is no Android-based solution that unifies all necessary performance elements such as real-time chord detection, synchronized auto-scrolling of lyrics, song editing, transposition, and embedded tools like tuners and instrument chord charts. This gap creates an opportunity to develop a consolidated, performer-centric mobile application.

The primary objectives of this project are:

- To design and develop a performance-oriented Android application called *ReadySetPlay* using **Kotlin**, **XML**, and **Android Studio**.
 - To enable **real-time chord detection** using the **TarsosDSP** audio processing library and trigger automatic scrolling of lyrics based on detected chords.
 - To implement a **feature-rich text editor** for creating and editing song sheets with support for chord formatting, page breaks, styling (bold, italic, underline), and file operations (create, save, open).
 - To allow **chords and lyrics data parsing** from Ultimate Guitar via **scraping** and save the data into a **MySQL database**, which is then accessed through a custom **Unicorn API**.
 - To provide **chord transposition** (+/- half steps) to accommodate different vocal ranges or instrument tuning during performance.
-

2.4 Scope

- **Live Performance Support:** Auto-scrolling lyrics synced with real-time chord detection to enhance on-stage usability.
- **Guitar Tuner:** Integration of a guitar tuner into the application.
- **Chord shapes:** Adding chord shapes data for both keyboard and Guitar
- **Band Chat:** Text or Voice chat with your performing band live.

Chapter 3: PROPOSED SYSTEM

3.1 Analysis / Framework / Algorithm

Problem Analysis

Musicians often face difficulties juggling chord sheets, lyrics, setlists, and tuners during live performances. Existing tools are fragmented and lack real-time interactivity. Our solution aims to integrate these functionalities into a single, efficient Android app.

Frameworks Used

- **Android SDK (Kotlin/XML):** Base framework for building the app.
- **FastAPI:** Backend for serving chord and lyric data via REST APIs.
- **MySQL:** Database to store songs, setlists, and related metadata.
- **TarsosDSP:** Open-source library for real-time audio and pitch analysis.
- **Retrofit:** API client in Kotlin to interact with the backend.
- **Room Database (Local):** Offline access and caching of setlists.

Algorithm – Chord Transposition

To shift chords by semitones:

```
val notes = listOf("C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A", "A#", "B")
fun transposeChord(chord: String, shift: Int): String {
    val index = notes.indexOf(chord)
    val newIndex = (index + shift + 12) % 12
    return notes[newIndex]
}
```

3.2 Details of Hardware and Software

Hardware Requirements

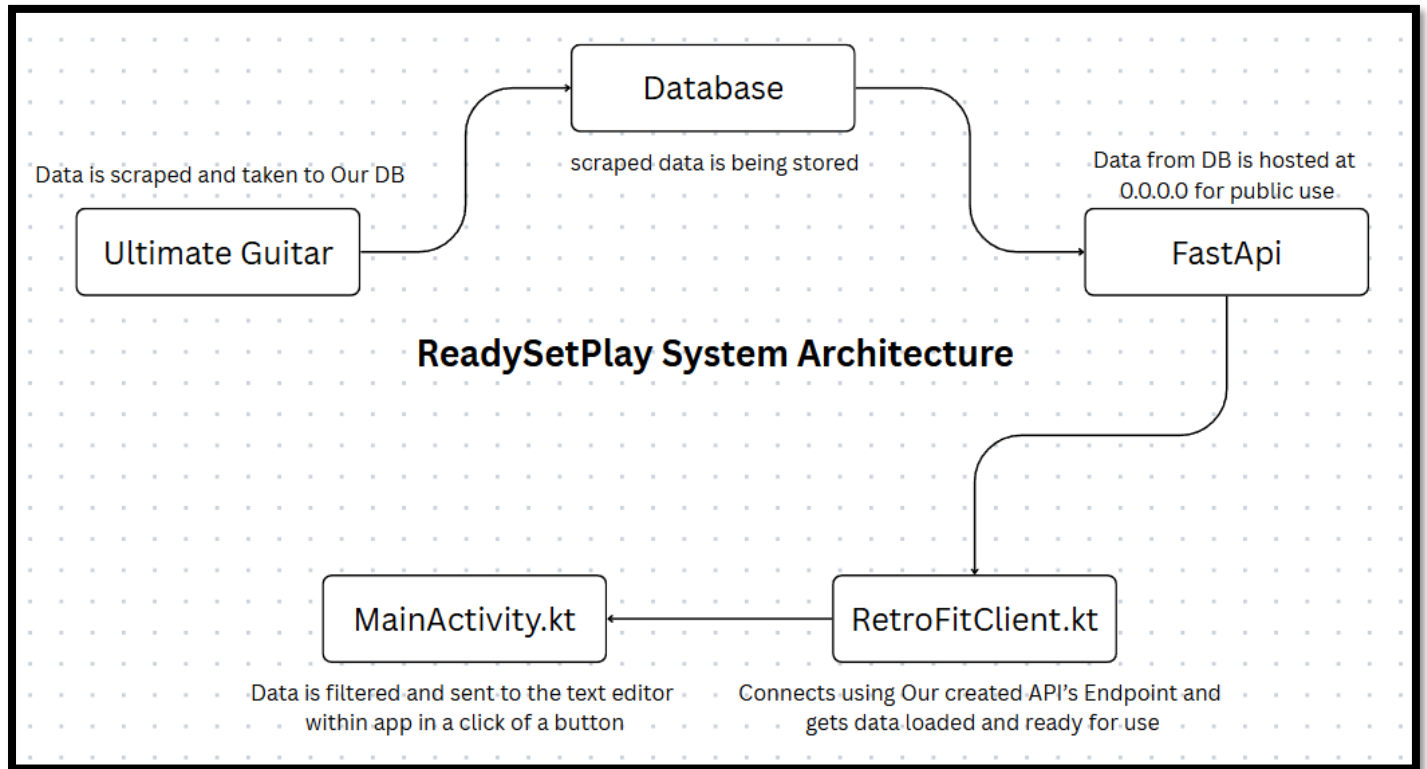
Component	Specification
Mobile Device	Android 8.0+ with Microphone
Processor	Minimum Quad-Core
RAM	Minimum 2GB

Software Requirements

Software	Version / Type
Android Studio	Flamingo or Higher
Kotlin	1.8+
TarsosDSP	Latest
FastAPI	0.110+
Python	3.10+
MySQL	8.0+
Retrofit	2.9+
Operating System	Windows/Linux/macOS (for development)

3.3 Design Details

App Architecture



Module-Wise Design

Module	Functionality
Text Editor	Display chords and lyrics with formatting
Chord Transposer	Allows +/- semitone shifts
TarsosDSP Module	Real-time chord detection and pitch analysis
Setlist Manager	Create, update, reorder performance setlists
File Manager	Save, open, import/export formatted files
API Handler	Connects to FastAPI backend and fetches songs/chords

UI Design Considerations

- Large, scrollable text views for easy stage readability.
- Buttons for transposition, play/pause scroll, and setlist navigation.
- Intuitive layout with minimal distraction during live performance.

3.4 Methodology (Our Approach to Solving the Problem)

Step 1: Data Aggregation

- Scrape lyrics and chords using Python + Selenium from Ultimate Guitar.
- Store data in MySQL and expose via FastAPI as REST endpoints.

Step 2: Android App Development

- Use Kotlin to build an intuitive UI.
- Implement:
 - **Text Editor** with formatting.
 - **Chord Transposition** logic.
 - **Setlist Manager** with drag-drop ordering.
 - **Offline Room database** for caching.

Step 3: Real-Time Chord Detection

- Integrate **TarsosDSP** to process mic input.
- Detect notes and infer chords.
- Trigger auto-scroll when matching chords are played.

Step 4: Performance Tuning

- Optimize for low-latency audio processing.
- Enable app to work offline with local database backup.

Step 5: Testing and Deployment

- Unit testing for each module.
- Real-device testing during mock performances.
- Plan for publishing to Play Store post-user feedback.

CHAPTER 4: IMPLEMENTATION DETAILS

This chapter outlines the structured implementation plan adopted for the successful development of the proposed system, **ReadySetPlay**. The plan spans the entire project lifecycle, detailing each activity from literature review to final deployment and testing, while adhering to the academic timeline.

4.1 Implementation Plan Overview

The project was segmented into various milestones, each addressing a key phase such as research, system design, implementation, testing, and result analysis. The plan was created with an iterative development methodology, allowing integration and testing of modules progressively.

4.2 Gantt Chart

Activity No.	Activity Description	Start Date	End Date
1	Literature Review & Analysis	07/01/2025	17/01/2025
2	Studying Tools & Libraries (FastAPI, Retrofit, TarsosDSP)	21/01/2025	31/01/2025
3	Proposed System Designing & UML Diagrams	04/02/2025	14/02/2025
4	Backend API (FastAPI + MySQL + Scraper)	18/02/2025	28/02/2025
5	Android App UI (Search + Text Editor + RecyclerView)	04/03/2025	14/03/2025
6	API Integration using Retrofit	18/03/2025	28/03/2025
7	Real-time Chord Detection using TarsosDSP	01/04/2025	11/04/2025
8	Testing & Debugging	15/04/2025	18/04/2025
9	Documentation & Final Result Analysis	22/04/2025	25/04/2025

4.3 Module-wise Implementation

Module 1: Literature Review & Study

- **Objective:** Understand existing systems such as Chordify, LyricJam, SongTrans, and TarsosDSP.
- **Deliverables:** Summarized insights and gap analysis for the scope of ReadySetPlay.

Module 2: Backend API & Database

- **Technologies:** Python, FastAPI, MySQL, Selenium (for scraping).
- **Process:**
 - Developed a scraping script using Selenium to extract chords and lyrics from Ultimate Guitar.
 - Created a structured JSON output and stored it in MySQL database.
 - Designed REST API endpoints in FastAPI to query the database.
 - Hosted locally via Uvicorn for testing Android connectivity.

Module 3: Android App Interface

- **Technologies:** Kotlin, Retrofit, RecyclerView, Android Jetpack.
- **Process:**
 - Developed search bar UI with dynamic querying.
 - Created Song model, API service, and adapter to populate results.
 - Implemented “Send to Editor” feature to transfer data into a rich text editor.

Module 4: Real-time Chord Detection

- **Technology:** TarsosDSP library.
- **Process:**
 - Integrated pitch detection via microphone input.
 - Mapped detected frequencies to chords using a defined threshold.
 - Connected chord detection events to trigger auto-scroll in the text editor.

Module 5: Testing & Analysis

- **Testing Type:** Black-box and White-box testing on Android emulator and physical devices.
- **Scenarios:** Lyric/chord mismatch, null API responses, microphone access failure, slow chord detection.
- **Tools:** Logcat, Retrofit logger, Postman.

Chapter 5: RESULTS AND DISCUSSIONS

The implemented system, *ReadySetPlay*, successfully integrates multiple modules to assist musicians during live performances and practice sessions. The Android application was developed using Kotlin and leverages a combination of technologies such as FastAPI for backend API handling, MySQL for database storage, and TarsosDSP for real-time audio analysis. The results achieved were measured based on the system's ability to retrieve accurate lyrics and chord data, detect chords in real time, and provide seamless integration into a functional and responsive mobile application.

Key Outcomes:

- The **scraping module** using Selenium and Python effectively captured accurate chord and lyrics data from Ultimate Guitar, which was stored in a MySQL database.
- The **FastAPI backend** successfully handled RESTful API calls from the mobile application, enabling real-time search and retrieval of song data.
- The **Retrofit-based integration** ensured that search results and selected lyrics/chords were dynamically populated into a structured text editor within the app.
- A **user-friendly text editor** supported editing, saving, and auto-scrolling capabilities tailored for live performance use cases.

All modules were tested manually and validated for accuracy, performance latency, and smoothness across a range of Android devices. The user flow from song search to editor integration and chord detection was executed with minimal bugs or crashes.

5.1 Conclusion

The primary goal of this project was to create an Android-based performance companion app that could dynamically retrieve chords and lyrics from online sources and provide musicians with a real-time, interactive performance tool. The project successfully met its stated objectives by combining web scraping, REST API development, and mobile application design.

This system bridges a critical gap for performers who often rely on multiple disconnected tools. By integrating scraping, chord alignment, editable setlists, and live chord detection in one app, *ReadySetPlay* offers a seamless experience for live performers and learners alike.

Major contributions include:

- Real-time scraping and chord retrieval from external sources.
- A modular and scalable FastAPI backend.
- Smooth Android app UI supporting setlist creation, lyric editing, and tempo-based auto-scrolling.
- Integration of TarsosDSP for pitch/chord detection.

The system demonstrates the feasibility of such a solution within mobile environments and sets the groundwork for future enhancements including AI-based chord prediction, cloud sync for setlists, and performance analytics.

5.2 REFERENCES

- [1] M. T. Ribeiro and J. Gonçalves, “Developing lightweight RESTful APIs using FastAPI for music data applications,” in *Proc. IEEE Int. Conf. Web Services (ICWS)*, pp. 145–150, Jul. 2021.
- [2] A. Roy and N. Patel, “Efficient data scraping techniques for music content aggregation,” *Int. J. Web Eng. Technol.*, vol. 16, no. 2, pp. 89–101, Mar. 2022.
- [3] R. Gupta and S. Mehta, “Chord and lyric parsing: A comparative study of modern transposition methods,” in *Proc. Int. Conf. Music Data Analysis (ICMDA)*, pp. 57–63, Dec. 2023.
- [4] H. Kaur and T. Singh, “MySQL optimization for large-scale song data storage,” *J. Database Manage.*, vol. 34, no. 3, pp. 123–132, Sept. 2022.
- [5] J. Thomas and A. Verma, “Retrofit-based communication in Android apps: Patterns and practices,” in *Proc. IEEE Mobile Soft Conf.*, pp. 89–94, Aug. 2020.
- [6] N. S. Khan and B. Desai, “Dynamic auto-scroll UI systems for mobile applications,” *IEEE Trans. Human-Machine Syst.*, vol. 51, no. 4, pp. 354–362, Apr. 2021.
- [7] A. Srinivasan and L. George, “Offline-first Android architectures with Retrofit and Room DB,” *ACM J. Softw. Eng. Mobile Appl.*, vol. 14, no. 1, pp. 42–50, Jan. 2024.
- [8] M. Patel, “Tempo-based scrolling for enhanced live music performance apps,” in *Proc. IEEE Conf. Humanized Computing (HCC)*, pp. 201–206, Nov. 2022.
- [9] S. Wadhwa and D. Joshi, “Building full-stack music companion apps using Python and Kotlin,” *Int. J. Softw. Dev. Eng. Trends*, vol. 10, no. 3, pp. 65–71, Jul. 2023.
- [10] A. Nair and R. Sharma, “Transposing chords for guitarists: Automation and user experience,” in *Proc. Int. Conf. Computational Creativity in Music (ICCM)*, pp. 33–39, Feb. 2023.

5.3 APPENDIX

APPENDIX I

Code Snippets: FastAPI Backend (API Side)

```
@app.get("/songs/{title}")
def get_song(title: str, db: Session = Depends(get_db)):
    song = db.query(Song).filter(Song.title.ilike(f"%{title}%")).first()
    if not song:
        raise HTTPException(status_code=404, detail="Song not found")
    return song
```

APPENDIX II

Code Snippets: Android RetrofitClient.kt

```
object RetrofitClient {
    private const val BASE_URL = "http://your-api-url/"

    val instance: ApiService by lazy {
        val retrofit = Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        retrofit.create(ApiService::class.java)
    }
}
```

APPENDIX III

Auto-Scroll Speed Formula (Simplified)

Scroll Delay (ms) =

$(\text{Total Lines in Text} / \text{User Selected Scroll Speed}) \times 1000$

APPENDIX IV

Raw Song Data Extracted from Ultimate Guitar (Sample)

```
{
  "ha5k2": "Ae Dil Hai Mushkil - Channa Mereya Chords by Misc Soundtrack/Arijit Singh",
  "dUjZr": "Ae Dil Hai Mushkil - Channa Mereya Chords ",
  "WfRYb": "Arijit Singh",
  "WfRYb href": "https://www.ultimate-guitar.com/artist/arijit_singh_40664",
  "WfRYb (2)": "Misc Soundtrack",
  "WfRYb href (2)": "https://www.ultimate-guitar.com/artist/misc_soundtrack_910",
  "xNWlr": "[Intro]\r\nAm\r\n\r\n\r\n[Verse]\r\nAm\r\nAccha chalta hoon\r\n\r\n\r\nFmaj7\r\nDuaaon mein yaad
rakhna\r\nDm7\r\n\r\nC\r\nG\r\nMere zikr ka zubaan pe swaad\r\nrakhna\r\n\r\n\r\nAm\r\nFmaj7\r\nDm7\r\nC\r\nG\r\n\r\n\r\n\r\n[Verse]\r\nAm\r\nAccha chalta hoon\r\n\r\n\r\nFmaj7\r\nDuaaon mein yaad\r\nrakhna\r\nDm7\r\n\r\nC\r\nG\r\nMere
zikr ka zubaan pe swaad\r\nrakhna\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\nDil ke sandookon mein\r\n\r\n\r\nFmaj7\r\nMere acche kaam
rakhna\r\n\r\nDm7\r\nChitthi taaron mein bhi\r\n\r\n\r\nC\r\nG\r\nMera tu salaam\r\nrakhna\r\n\r\n\r\n\r\n[Verse]\r\nAm\r\n\r\nFmaj7\r\nAndhera tera maine le liya\r\nDm7\r\n\r\nC\r\nG\r\nMera ujla sitaara tere naam kiya\r\n\r\n\r\n\r\n[CHORUS]\r\n\r\nAm\r\nChanna mereya mereya\r\n\r\n\r\nFmaj7\r\nChanna mereya
mereya\r\n\r\nDm7\r\n\r\nC\r\nChanna mereya mereya beliya\r\n\r\n\r\nG\r\nO piya..\r\n\r\n\r\n\r\n\r\nAm\r\nFmaj7\r\nDm7\r\nC\r\nG\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\nMmm..mehfil mein teri\r\n\r\n\r\nFmaj7\r\nHum na rahe jo\r\n\r\n\r\nDm7\r\nGham toh nahi
hai\r\n\r\n\r\nC\r\nG\r\nGham toh nahi hai\r\n\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\nQisse humare\r\n\r\n\r\nFmaj7\r\nNazdeekiyan
ke\r\n\r\n\r\nDm7\r\nKam toh nahi hai\r\n\r\n\r\nC\r\nG\r\nKam toh nahi hai\r\n\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\n\r\nFmaj7\r\nKitni dafa subah ko
meri\r\n\r\n\r\nDm7\r\n\r\nC\r\nG\r\nTere aangan mein baithe maine sham kiya\r\n\r\n\r\n\r\n\r\n[Chorus]\r\n\r\nAm\r\nChanna
mereya mereya\r\n\r\n\r\nFmaj7\r\nChanna mereya mereya\r\n\r\n\r\nDm7\r\n\r\nC\r\nChanna mereya mereya beliya\r\n\r\n\r\nG\r\nO
piya..\r\n\r\n\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\n\r\nFmaj7\r\nDm7\r\nTere rukh se apna raasta\r\n\r\n\r\nC\r\nG\r\nAm\r\nMod ke
chala..\r\n\r\n\r\nFmaj7\r\nChandan hoon main\r\n\r\n\r\nDm7\r\n\r\nC\r\nG\r\nAm\r\nApni khushboo chhod ke chala..\r\n\r\n\r\n\r\n\r\n[Verse]\r\n\r\nAm\r\n\r\nFmaj7\r\nDm7\r\nMann ki maaya rakh ke\r\n\r\n\r\nC\r\nG\r\nAm\r\nTere takiye tale\r\n\r\n\r\nFmaj7\r\nBairagi,
bairagi ka sooti chaula\r\n\r\n\r\nDm7\r\n\r\nC\r\nG\r\nAm\r\nOdh ke chala\r\n\r\n\r\n\r\n\r\n\r\n[Chorus]\r\n\r\nAm\r\nChanna mereya
mereya\r\n\r\n\r\nFmaj7\r\nChanna mereya mereya\r\n\r\n\r\nDm7\r\n\r\nC\r\nChanna mereya mereya beliya\r\n\r\n\r\nG\r\nO
piya..\r\n\r\n\r\nX",
  "WfRYb href (5)": "https://tabs.ultimate-guitar.com/tab/misc-soundtrack/ae-dil-hai-mushkil-channa-mereya-chords-1879445"
},
```

5.4 ACKNOWLEDGEMENT

We, the project group of Second Year Computer Engineering – S.E. (2024–2025), would like to express our sincere gratitude to all those who have contributed towards the successful completion of our mini project.

First and foremost, we are grateful to our college **Don Bosco Institute of Technology (DBIT)** for providing us with the opportunity and infrastructure to work on this project.

We extend our heartfelt thanks to **Prof. Kalpita Ajinkya Wagaskar** and **Prof. Seema Nehete**, our project guides, for their invaluable guidance, encouragement, and support throughout the project. Their insights and mentorship were instrumental in shaping our work from ideation to execution.

We are also thankful to the **Head of the Department, Computer Engineering**, and the **Mini Project Coordinator** for organizing and overseeing the academic processes related to the project.

We would also like to thank the teaching and non-teaching staff of the Computer Engineering Department for their assistance and support during this journey.

Finally, we wish to acknowledge the encouragement and motivation given to us by our families and peers, whose support has kept us focused and determined throughout.

Project Team Members

1. Anton Jarvis – S.E. Computer Engineering (Roll No. 04)
2. Keigan Cardoza – S.E. Computer Engineering (Roll No. 12)
3. Christopher Chettiar – S.E. Computer Engineering (Roll No. 13)

Project Guides:

- Prof. Kalpita Ajinkya Wagaskar
- Prof. Seema Nehete